Java-проект

Сборка и зависимости

Maven, библиотеки и первый проект



От исходников к приложению

- Из чего состоит Java-проект
- Как Maven собирает код и подключает библиотеки
- Где хранятся зависимости и как их найти
- Что получается после сборки и как это запустить

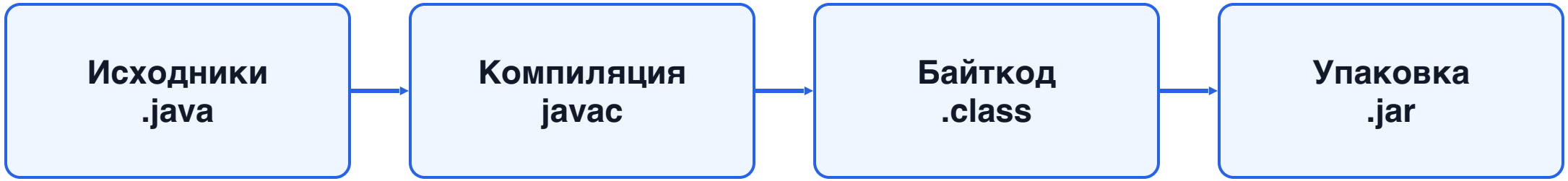
После рассказа соберём проект и подключим Jackson.

Один файл постепенно становится проектом

- Классы приложения распределены по пакетам
- Ресурсы содержат настройки и другие данные
- Тесты проверяют поведение кода
- Конфигурация сборки описывает зависимости и действия

Проект содержит всё необходимое для разработки и сборки.

Как появляется исполняемый код



JVM исполняет классы. JAR объединяет классы и ресурсы.

Что пришлось бы делать вручную

- Найти исходники и вызвать компилятор
- Скачать нужные библиотеки и указать пути к ним
- Запустить тесты и скопировать ресурсы
- Упаковать результат и повторить действия после изменений

Инструмент сборки автоматизирует эту последовательность.

Кто за что отвечает

Инструмент	Роль
JDK	Компилятор, JVM и инструменты Java
Maven	Организация сборки и работа с зависимостями
IDE	Редактирование, навигация, запуск и отладка
Git	История изменений исходников и настроек

Проект можно собрать из терминала, без открытой IDE.

Maven и другие инструменты сборки

- Apache Maven описывает проект в pom.xml
- Gradle использует сценарии сборки на Kotlin или Groovy
- Оба инструмента умеют подключать библиотеки и запускать тесты
- В учебных примерах используем Maven и его стандартную структуру

Выбор сборщика не меняет синтаксис Java.

Структура Maven-проекта

Путь	Содержимое
pom.xml	Описание проекта и сборки
src/main/java	Java-код приложения
src/main/resources	Ресурсы приложения
src/test/java	Код тестов
target	Результаты сборки

Пакет course.tasks соответствует папкам course/tasks внутри java.

pom.xml: описание нашего проекта

Минимальное описание проекта

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>ru.example.course</groupId>
  <artifactId>hello-maven</artifactId>
  <version>1.0.0</version>
</project>
```

POM означает
Project Object Model.

XML-теги описывают
параметры проекта.

4.0.0 в modelVersion обозначает модель POM, а не установленный Maven.

Координаты артефакта

Артефакт — файл, который создают или распространяют, например JAR

Часть	Пример	Назначение
groupId	ru.example.course	Группа или пространство имён
artifactId	hello-maven	Имя конкретного артефакта
version	1.0.0	Версия артефакта

Координаты есть и у нашего проекта, и у подключаемых библиотек.

Версия Java в настройках сборки

Фрагмент pom.xml внутри project

```
<properties>
  <maven.compiler.release>17</maven.compiler.release>
  <project.build.sourceEncoding>
    UTF-8
  </project.build.sourceEncoding>
</properties>
```

release задаёт
целевую версию Java.

UTF-8 задаёт
кодировку исходников.

JDK, на котором работает Maven, проверяем командой mvn -v.

Библиотека: готовый код для нашей программы

- Предоставляет классы и методы для определённых задач
- Наш код вызывает её публичный API
- В Java библиотеку часто распространяют в JAR
- Например, Jackson читает и создаёт JSON

Библиотека позволяет использовать готовую функциональность.

Библиотека и фреймворк

	Библиотека	Фреймворк
Что предоставляет	Операции для конкретной задачи	Правила и механизмы приложения
Кто вызывает код	Наш код вызывает библиотеку	Фреймворк вызывает наш код
Пример	Jackson преобразует объект в JSON	Spring вызывает обработчик запроса

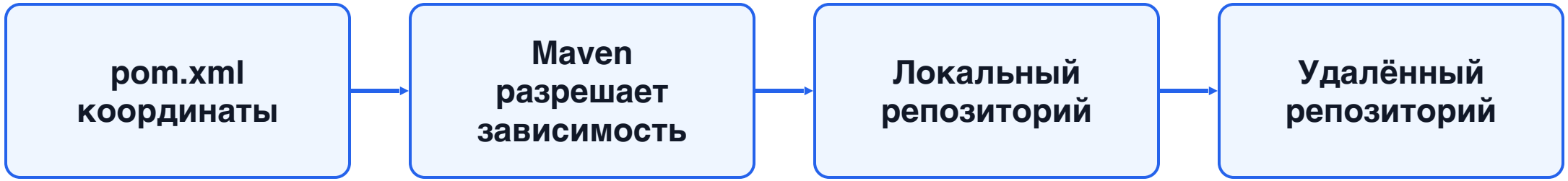
Это полезный ориентир. Между категориями нет жёсткой границы.

Maven и Maven Central

Название	Что это
Apache Maven	Программа для сборки проекта
Maven Central	Удалённый репозиторий артефактов
Локальный репозиторий	Каталог с загруженными и установленными артефактами
Корпоративный репозиторий	Хранилище внутренних библиотек и внешних артефактов

Другие инструменты сборки тоже могут загружать библиотеки из Central.

Откуда берётся библиотека



Недостающие артефакты Maven загружает и сохраняет локально.

Два смысла слова «репозиторий»

	Git-репозиторий	Репозиторий артефактов
Что хранит	Файлы проекта и их историю	Версии JAR, POM и других файлов
Как выбираем	Ветка, тег, коммит	Координаты и версия
Зачем нужен	Разрабатывать и обсуждать изменения	Подключать результаты сборки

Один проект может иметь исходники в Git и релизы в Central.

Подключение Jackson

Фрагмент pom.xml внутри project

```
<dependencies>
  <dependency>
    <groupId>tools.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>3.1.0</version>
  </dependency>
</dependencies>
```

dependencies содержит зависимости проекта.

dependency описывает одну библиотеку.

Версию фиксируем явно, чтобы группа использовала одинаковый пример.

Прямые и транзитивные зависимости



Maven читает POM библиотек и подключает их зависимости.

import и зависимость решают разные задачи

Объявление import и фрагмент main

```
import tools.jackson.databind.json.JsonMapper;

JsonMapper mapper = JsonMapper.builder().build();
```

Зависимость делает библиотеку доступной проекту.

import позволяет писать короткое имя класса.

Один import не скачает библиотеку и не подключит её к сборке.

Где нужна зависимость: scope

Scope	Когда доступна	Пример назначения
compile	При компиляции и выполнении	Jackson в коде приложения
test	При компиляции и запуске тестов	JUnit в тестах
runtime	При выполнении и в тестах	Драйвер БД, подключаемый во время запуска

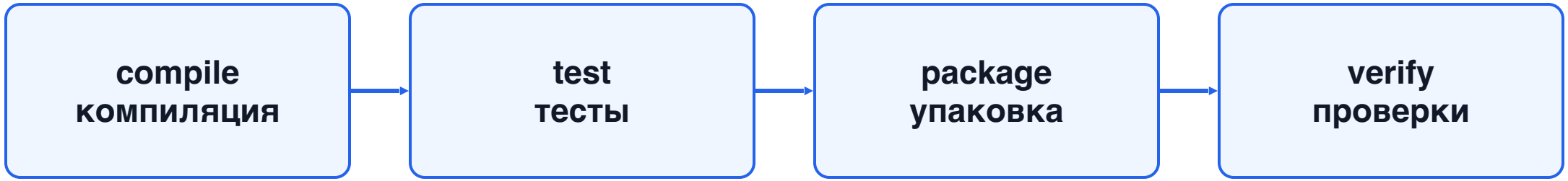
Без указанного scope Maven использует compile.

Зависимость и плагин

	Зависимость	Плагин Maven
Зачем	Предоставляет API нашему коду	Выполняет действие сборки
Пример	jackson-databind	maven-compiler-plugin
Где в POM	dependencies	build / plugins

Плагины компилируют код, запускают тесты и упаковывают JAR.

Основные фазы сборки



mvn package выполняет предшествующие фазы и доходит до упаковки.

Фаза и цель плагина

Команды терминала из каталога с pom.xml

```
mvn package

mvn dependency:tree

mvn clean package
```

package — фаза.

dependency:tree — цель плагина.

clean относится к отдельному циклу.

mvn package сам по себе не запускает clean.

package, install и deploy

Команда	Результат
mvn package	Создаёт артефакт в target
mvn install	Также помещает артефакт в локальный репозиторий
mvn deploy	Также публикует артефакт в настроенный удалённый репозиторий

deploy в Maven означает публикацию артефакта, а не запуск сервера.

Что находится внутри JAR

- JAR — архив на основе ZIP
- Обычно содержит .class и ресурсы приложения
- META-INF/MANIFEST.MF хранит служебную информацию
- Обычная упаковка не добавляет все JAR зависимостей внутрь

Содержимое можно посмотреть командой jar tf файл.jar.

Classpath: где JVM ищет классы

Запуск простого проекта без внешних зависимостей

```
java -cp target/hello-maven-1.0.0.jar \
  course.Main
```

В classpath указываем каталоги и JAR.

course.Main — класс с методом main.

JVM нужны классы приложения и его библиотек.

Запуск JAR: что должно быть настроено

Вариант	Что нужно
java -cp … course.Main	Classpath и имя класса с main
java -jar app.jar	Main-Class в манифесте и доступные зависимости
JAR с зависимостями	Отдельная настройка упаковки или загрузчика

Для автономного запуска нужна соответствующая упаковка.

Одинаковая версия Maven: Wrapper

Команды проекта с добавленным Maven Wrapper

```
./mvnw -v
./mvnw package

# Windows
mvnw.cmd package
```

Wrapper хранится в проекте.

Он запускает заданную версию Maven.

Wrapper фиксирует Maven. Версию JDK согласовываем отдельно.

Типичные проблемы первого запуска

Симптом	Что проверить
Не находится библиотека	Координаты, версию, доступ к репозиторию
release version … not supported	Версию JDK в выводе mvn -v
Класс есть в IDE, но не при запуске	Зависимости в runtime classpath
no main manifest attribute	Способ запуска и Main-Class в манифесте

Читаем причину ошибки выше строки BUILD FAILURE.

JSON: данные в текстовом формате

Пример JSON

```
{
  "title": "Изучить Maven",
  "done": false,
  "tags": ["java", "backend"]
}
```

{ } обозначают объект.
[ ] обозначают массив.

Имена полей и строки в двойных кавычках.

false — логическое значение.

JSON описывает данные, а не методы Java-объекта.

Сериализация и десериализация



Сериализация создаёт представление данных. Десериализация читает его.

Какие операции покажем в проекте

Краткий обзор API Jackson перед демонстрацией

```
JsonMapper mapper = JsonMapper.builder().build();

String json = mapper.writeValueAsString(task);

Task restored = mapper.readValue(json, Task.class);
```

writeValueAsString создаёт JSON-строку.

readValue читает её в указанный тип.

Task.class сообщает библиотеке, какой объект нужно получить.

Материалы для повторения

- Maven: стандартная структура и POM Reference
- Maven: жизненный цикл, зависимости и репозитории
- Jackson: примеры преобразования объектов в JSON
- Java: команды java и jar

Ссылки на конкретные разделы находятся в заметках к слайдам.

Демонстрация: Maven и Jackson

- Создадим hello-maven и запустим Main
- Соберём JAR и запустим его из терминала
- В tasks-json подключим Jackson и посмотрим зависимости
- Преобразуем Task в JSON и восстановим объект

Теперь переходим к проектам в IDE и терминале.